

Design and Implementation of a Multi-GPU Macro-Dataflow Runtime System

Romain PEREIRA, Swann PERARNAU, Mathematics and Computer Science Division, Argonne National Laboratory
Pierre-Etienne POLLET, Thierry GAUTIER, Inria, LIP, Lyon, France

ABSTRACT

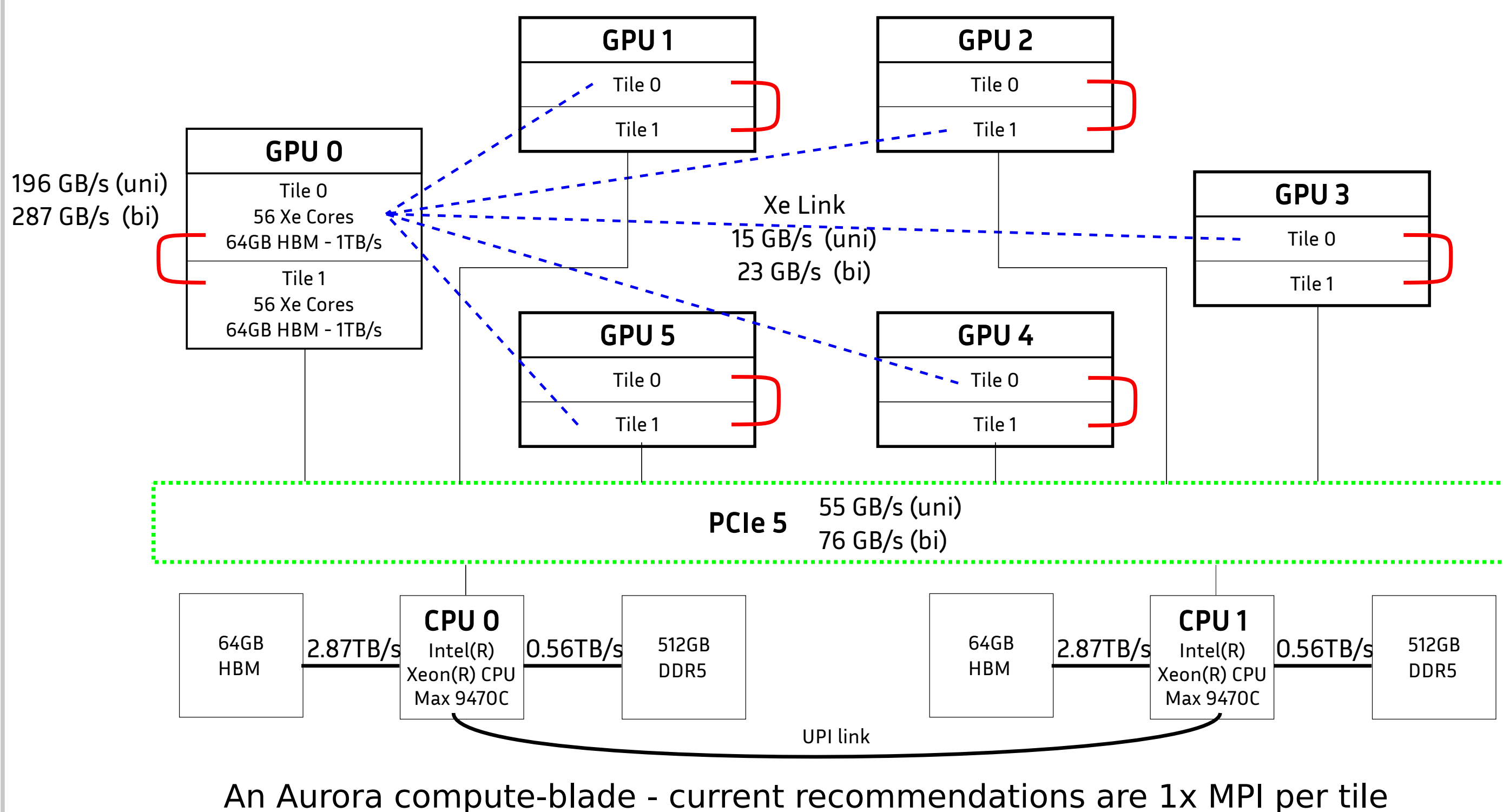
Multi-GPU systems dominate the HPC and AI computing facilities due to their energy efficiency [1]. The variety of vendors and hardware complexity creates a challenge in conceiving performing and portable applications.

Macro data-flow task-based systems are a promising solution to meet this challenge, delegating the responsibility from user-code to architecture-aware compilers and runtime systems. Workloads are split into dependent tasks and distributed on available processors, implicitly overlapping computation with transfers following tasks accesses data-flow [2].

This poster introduces a fork of Kaapi [3] for macro-dataflow and task-based programming on multi-gpu architectures with support for AMD/Intel/Nvidia GPUs and networks (nvlLink / xe-link). We present the runtime system's internal abstractions towards achieving performance portability, an extension for 2-dimensional task accesses, and illustrate it with XKBlas: a multi-GPU BLAS library.

TARGETING MULTI-GPU COMPUTE NODES

Towards a **single** UNIX process on multi-GPU compute blades



DATAFLOW OVER 2-DIMENSIONAL REGIONS

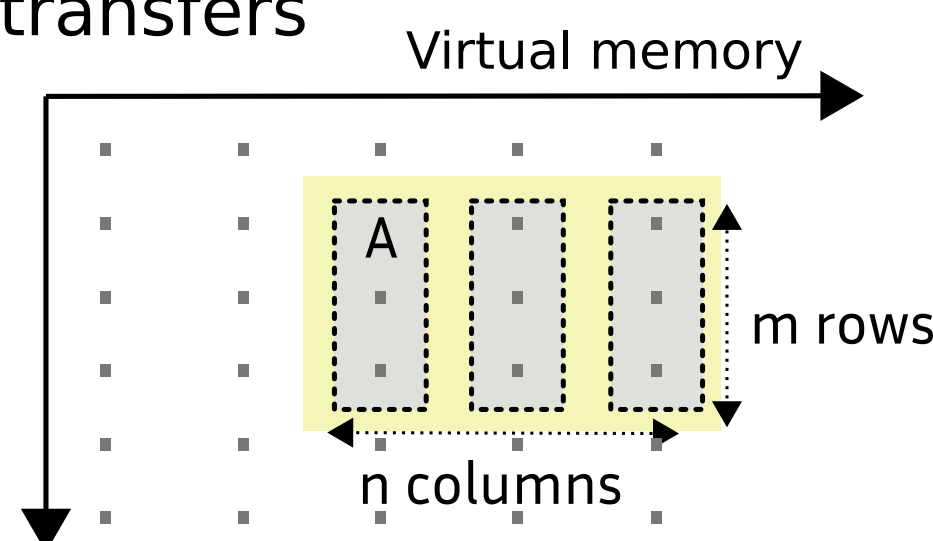
Adds run-time management overhead against state-of-the-art [2,7]

- 1) Conflicts detection over 2 dimensional memory regions
- 2) Coherency between memories minimizing data transfers

BLAST matrix descriptor [4]

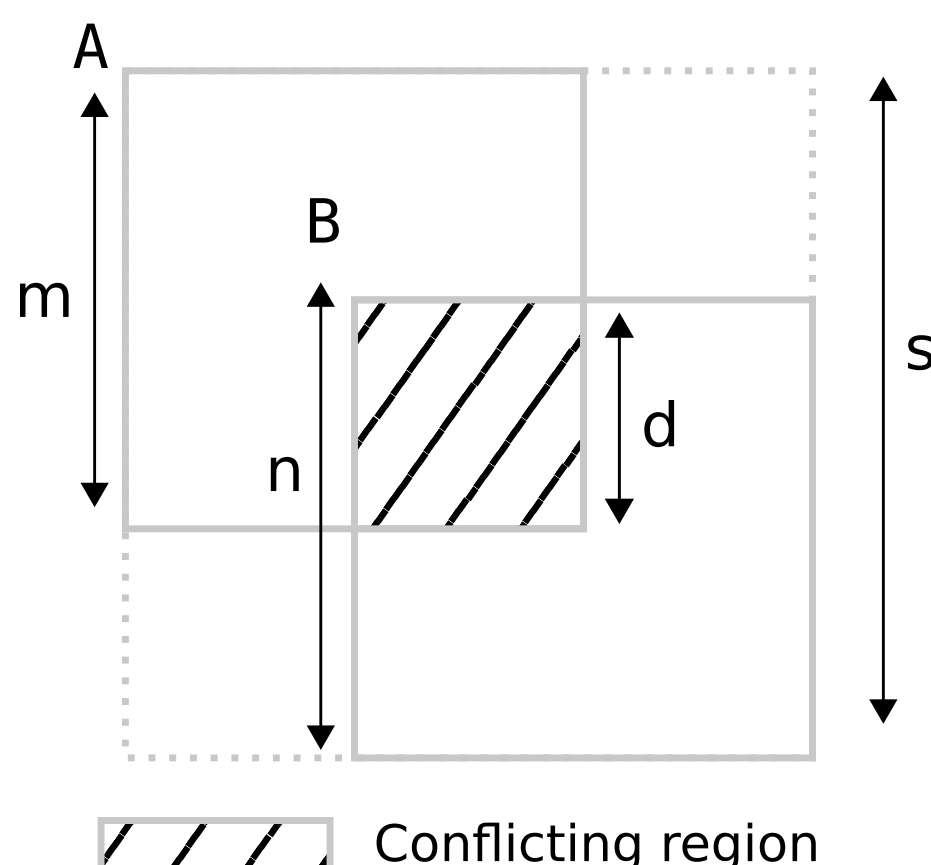
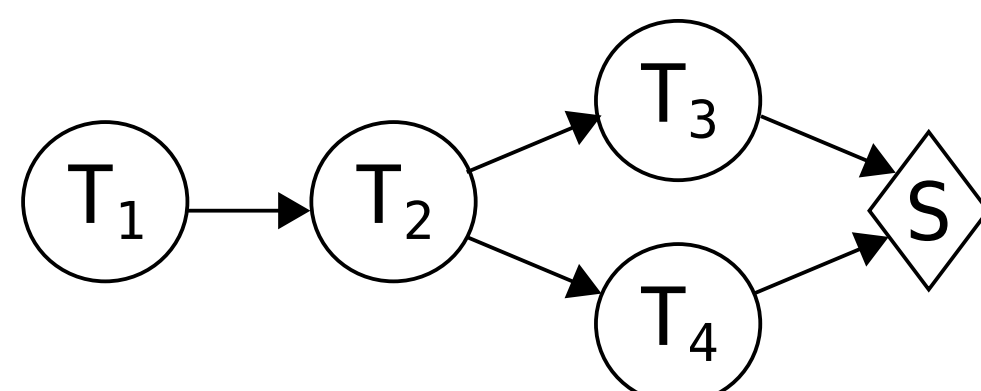
$$M = (A, m, n, L_D)$$

$$mem(M) = \bigcup_{i \in [0..n]} [A + i \cdot L_D \dots A + i \cdot L_D + m]$$



Implicit data-flow based on accesses

T1: write(A, m, m, LD)
T2: write(B, n, n, LD)
T3: read (A, s, s, LD)
T4: read (B, d, d, LD)
S: sync



CONCLUSIONS AND FUTURE WORKS

Performance portability across AMD/Nvidia multi-gpu systems.
Intel system (Aurora) still experimental.
Low-overhead on 2-dimensional accesses management.

Extending to n-dimensional accesses for stencil scientific simulation.
Power-aware communication/computation scheduling.
OpenMP extensions for implicit data-flow and multi-GPU execution.

DESIGN

* experimental

Application	MUMPS				
Specialization	XKBlas	XKBM		XKOMP*	
Runtime	Kaapi Runtime System				
Driver	pthread, hwloc	SYCL, MKL, Level Zero	cuBLAS, Cuda	rocBLAS, HIP	ciBLAST, OpenCL

User Thread application control-flow $\{ \{ \dots \} \}$ x threads

Tasks with data accesses $\text{gemm read } A[0:m][0:k], B[0:k][0:n] \text{ write } C[0:m][0:n]$ y tasks

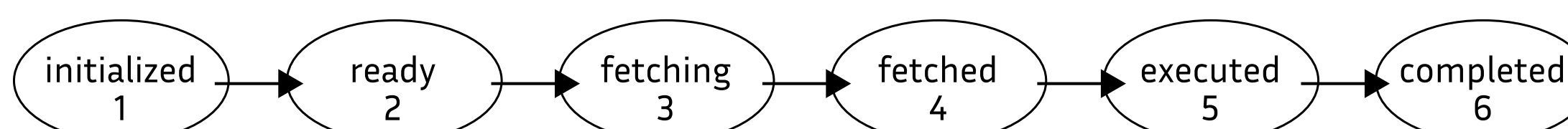
Device a driver-device handle CPU ... GPU n devices

Thread pthread bound topologically $\{ \{ \dots \} \}$ m threads per device

Stream queues of instructions with a driver-stream H2D D2H D2D ... Kern k streams per thread

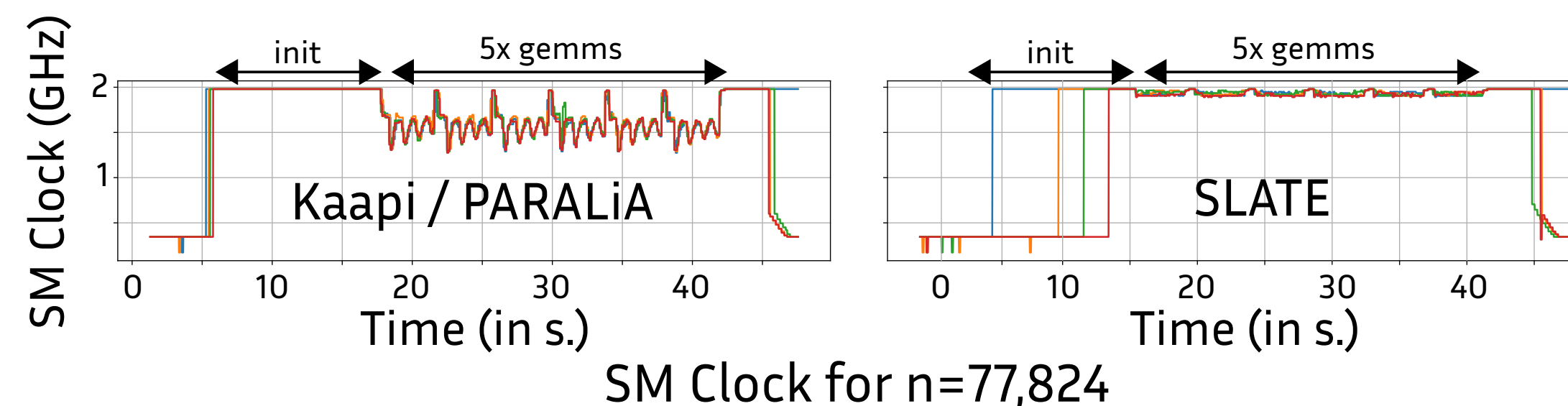
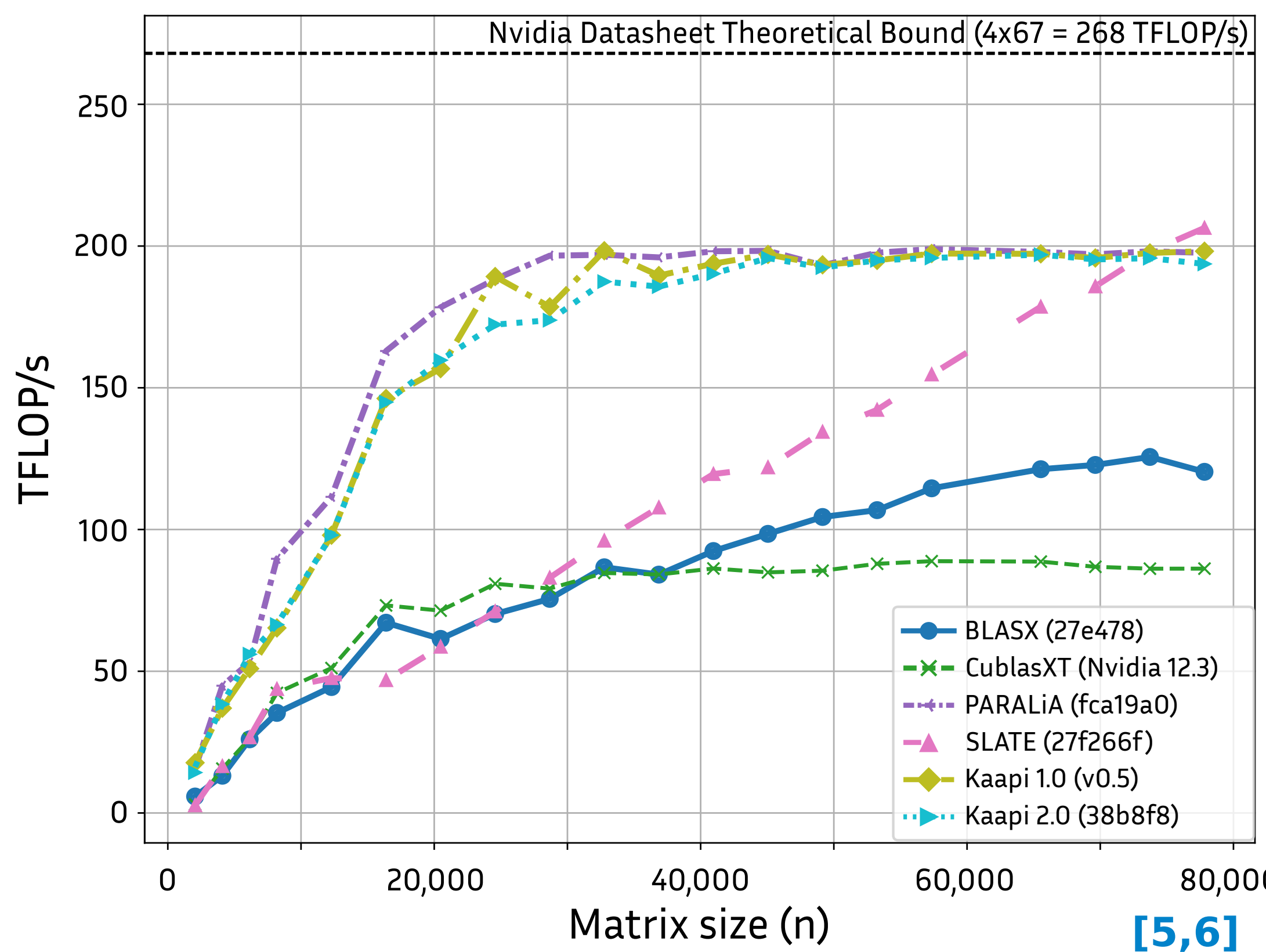
Tasks transition system

Execution model based on a 6-state task system



MULTI-GPU BLAS LIBRARIES

FP64 GEMM performances on 4x H100-SXM GPU (700W) and an Intel(R) Xeon(R) Platinum 8468 CPU.



REFERENCES

- [1] Papatheodore, Tom. OLCF's Frontier Supercomputer. ATPESC Jul. 2023
- [2] Bauer, Michael. Legion: programming distributed heterogeneous architectures with logical regions. PhD. Thesis (2014)
- [3] Gautier, Thierry et al. KAAPI: A thread scheduling runtime system for data flow computations on cluster of multi-processors . PASCO '07
- [4] Basic Linear Algebra Subprograms Technical (BLAST) Forum. Basic Linear Algebra Subprograms (BLAS) Technical Forum Standard. Aug. 2021
- [5] Anastasiadis, Petros et al. Uncut-GEMMs: Communication-Aware Matrix Multiplication on Multi-GPU Nodes. CLUSTER'24.
- [6] Gates, Mark et al. Evolution of the SLATE linear algebra library. The International Journal of High Performance Computing Applications (2025)
- [7] Slaughter, Elliott et al. Task Bench: A Parameterized Benchmark for Evaluating Parallel Runtime Performance. Supercomputing (SC) 2020